

Deployment — Circles (Azure App Service)

Circles består av två deploybara appar som körs som separata **Azure App Services (Linux, .NET 10)**:

App	Projekt	Startkommando (Azure)
circles-web-dev	src/Circles.Web/ Circles.Web.csproj	dotnet Circles.Web.dll
circles-api-dev	src/Circles.API/ Circles.API.csproj	dotnet Circles.API.dll

Deploy sker automatiskt via GitHub Actions vid push till `main` (workflow-filerna ligger i `.github/workflows/`).

1. Startup Command (viktigast — löser “Azure default-sidan”)

På **Linux App Service** måste startkommandot sättas explicit, annars vet Azure inte vilken DLL som är entry point och visar sin default-sida.

Sätt i portalen under **App Service → Configuration → General settings → Startup Command**:

- **circles-web-dev:** `dotnet Circles.Web.dll`
- **circles-api-dev:** `dotnet Circles.API.dll`

Detta krävs eftersom en publish-mapp kan innehålla flera `.dll`-filer. Startkommandot pekar ut rätt applikations-DLL. GitHub Actions-workflowen publicerar numera bara rätt projekt (`dotnet publish src/Circles.Web/Circles.Web.csproj ...`), men startkommandot ska ändå sättas för att vara explicit och robust.

2. App Settings (miljövariabler) som MÅSTE sättas

Sätt under **App Service → Configuration → Application settings** för **båda** apparna:

Namn	Värde	Kommentar
ASPNETCORE_ENVIRONMENT	Production	Väljer appsettings.Production.json
ConnectionStrings__Circles	(se format nedan)	Dubbelt understreck = nästlad config-nyckel

Dubbelt understreck (__) är hur Azure/.NET mappar en miljövariabel till en nästlad konfigurationsnyckel. `ConnectionStrings__Circles` motsvarar `ConnectionStrings:Circles` i `appsettings.json`.

Alternativt kan connection stringen sättas under **Configuration → Connection strings** (typ: **SQLAzure**) — då exponeras den som `ConnectionStrings__Circles` automatiskt.

Appen läser connection stringen i denna prioritetsordning (se `Program.cs`):

1. `ConnectionStrings:Circles` (dvs. `ConnectionStrings__Circles`)
2. `AZURE_SQL_CONNECTIONSTRING` (Azures standardnamn om du använder deras SQL-integration)
3. En lokal SQL Server-fallback (endast för lokal utveckling)

Tomma/whitespace-värden behandlas som “inte satt”.

API:et behöver dessutom

Namn	Värde	Kommentar
<code>Auth__JwtSigningKey</code>	(en hemlig nyckel, minst 32 tecken)	Krävs i produktion för JWT-signering

E-postnotifieringar (SMTP) — valfritt men rekommenderat i produktion

Circles skickar e-postnotiser vid nya meddelanden, nya diskussioner och när en uppgift tilldelas en medlem. E-post skickas via SMTP (MailKit) och konfigureras via App Settings. Sätt dessa för **båda** apparna (webben skickar notiser vid åtgärder i UI:t; API:et vid åtgärder via API:et):

Namn	Exempelvärde	Kommentar
Email__SmtpHost	smtp.sendgrid.net	SMTP-serverns värdnamn. Lämnas tomt = e-post stängs av (ingen notis skickas, inget fel kastas)
Email__SmtpPort	587	587 = STARTTLS (standard), 465 = implicit TLS
Email__Username	apikey	SMTP-användarnamn (för SendGrid är det ordagrant apikey)
Email__Password	(hemlig nyckel/lösenord)	SMTP-lösenord eller API-nyckel — checka aldrig in i git
Email__FromAddress	noreply@circles.app	Avsändaradress (bör vara en verifierad domän hos din leverantör)
Email__FromName	Circles	Visningsnamn för avsändaren

Graceful fallback: om Email__SmtpHost är tomt loggas en varning och notiserna hoppas tyst över — appen fungerar normalt utan e-postkonfiguration (bra för dev/test). Notiser skickas dessutom “fire-and-forget”: ett SMTP-fel stoppar aldrig själva åtgärden (att publicera ett meddelande, starta en diskussion eller tilldela en uppgift).

Leverantörer: valfri SMTP-tjänst fungerar. Vanliga val på Azure är **SendGrid** (smtp.sendgrid.net:587 , användarnamn apikey) eller **Azure Communication Services** (Email). Endast aktiva medlemmar med ett kopplat konto (e-postadress) får notiser.

3. Connection string-format för Azure SQL

Rekommenderat: passwordless med managed identity

Aktivera **System-assigned managed identity** på App Service och ge den db_datareader + db_datawriter (samt rätt att köra migrations, t.ex. db_ddladmin) i Azure SQL-databasen. Ingen hemlighet behöver då lagras:

```
Server=tcp:veumsql.database.windows.net,1433;Initial Catalog=circles-dev;Encrypt=True;TrustServerCertificate=False;Connection Timeout=30;Authentication=Active Directory Default;
```

Alternativ: SQL-autentisering (användarnamn/lösenord)

```
Server=tcp:veumsql.database.windows.net,1433;Initial Catalog=circles-dev;User
ID=<user>;Password=<password>;Encrypt=True;TrustServerCertificate=False;Connection
Timeout=30;
```

Ingen connection string med lösenord ska checkas in i git. `appsettings.Production.json` har därför en tom `ConnectionStrings:Circles` — det verkliga värdet sätts i Azure.

4. Migrering och seed

Båda apparna kör `Database.MigrateAsync()` + `DataSeeder.SeedAsync()` vid uppstart. Migreringar appliceras alltså automatiskt mot den konfigurerade databasen första gången en app startar mot den. Managed identity (eller SQL-användaren) behöver därför rättigheter att ändra schema (skapa tabeller) första gången.

5. Snabb checklista vid ny deploy

1. Startup Command satt för respektive app (`dotnet Circles.Web.dll` / `dotnet Circles.API.dll`).
2. `ASPNETCORE_ENVIRONMENT=Production` satt.
3. `ConnectionStrings__Circles` satt (eller Connection strings-sektionen).
4. `Auth__JwtSigningKey` satt (endast API:et).
5. `Email__*` satt om e-postnotiser ska skickas (annars hoppas de tyst över).
6. Managed identity aktiverad och har DB-rättigheter (om passwordless).
7. Push till `main` → GitHub Actions bygger och deployar.
8. Verifiera: `https://<app>.azurewebsites.net/health` (API) svarar, och webben visar inloggningssidan istället för Azure default-sidan.

6. Extern inloggning: Google + Microsoft (valfritt)

`Circles.Web` kan låta medlemmar logga in med Google eller Microsoft **utöver** e-post/lösenord. Inloggningen matchar bara mot **befintliga** konton — vi skapar aldrig nya `UserAccount` automatiskt. Om den externa e-postadressen inte finns som ett Circles-konto visas felmeddelandet
 “Inget Circles-konto hittades ... Kontakta din administratör.”

Knapparna visas bara för de leverantörer som är konfigurerade (tomt `ClientId` = dold knapp).

6.1 Google OAuth

1. Gå till `https://console.cloud.google.com/` → skapa/välj ett projekt.
2. **APIs & Services** → **OAuth consent screen**: välj “External”, fyll i appnamn (Circles), support-e-post och spara.

3. APIs & Services → Credentials → Create Credentials → OAuth client ID:

- Application type: **Web application**
- **Authorized redirect URIs** — lägg till exakt:
 - `https://<app>.azurewebsites.net/signin-google` (produktion/dev i Azure)
 - `https://localhost:7099/signin-google` (lokal utveckling)

4. Kopiera **Client ID** och **Client secret**.

6.2 Microsoft (Azure AD / Microsoft-konto)

1. Gå till `https://portal.azure.com` → **Microsoft Entra ID** → **App registrations** → **New registration**.
2. Namn: Circles. Supported account types: välj det som passar (t.ex. "Accounts in any organizational directory and personal Microsoft accounts").
3. **Redirect URI** (plattform: Web) — lägg till exakt:
 - `https://<app>.azurewebsites.net/signin-microsoft`
 - `https://localhost:7099/signin-microsoft`
4. Efter registrering: kopiera **Application (client) ID**.
5. **Certificates & secrets** → **New client secret** → kopiera secret-värdet.

6.3 App-inställningar (Azure App Service → Configuration)

Sätt följande (dubbelt understreck `__` = hierarki i .NET-konfiguration):

App Setting	Värde
<code>Authentication__Google__ClientId</code>	Google Client ID
<code>Authentication__Google__ClientSecret</code>	Google Client secret
<code>Authentication__Microsoft__ClientId</code>	Microsoft Application (client) ID
<code>Authentication__Microsoft__ClientSecret</code>	Microsoft client secret-värde

Lokalt kan samma värden läggas i `appsettings.Development.json` eller via `dotnet user-secrets`. Lämna tomt för att stänga av respektive knapp.

Viktigt: redirect-URI:erna ovan (`/signin-google` , `/signin-microsoft`) måste matcha exakt, inklusive https och domän, annars nekar leverantören återanropet.